

滑动窗口协议设计和实现实验报告

L^AT_EX by 王子轩 (学号: 2022211210)

2024 年 5 月 5 日

目录

1	实验内容和实验环境描述	3
1.1	实验任务和实验内容	3
1.2	实验环境	3
2	软件设计	3
2.1	带 NAK 帧通知的 SR 协议及优化	3
2.1.1	数据结构	3
2.1.2	模块结构	5
2.1.2.1	初代协议	5
2.1.2.2	优化协议	6
2.1.3	算法流程	6
2.2	分别确认和累计确认相结合的 SR 协议	7
2.2.1	数据结构	7
2.2.2	模块结构	8
2.2.3	算法流程	8
3	实验结果分析	9
3.1	程序正确性和健壮性	9
3.2	协议参数的选取	9
3.2.1	初代参数	9
3.2.1.1	滑动窗口大小	9
3.2.1.2	两种定时器超时参数	9
3.2.2	参数改进	10
3.2.2.1	基本模型	10
3.2.2.2	解决无意义的重传	10
3.2.3	最终优化	13
3.3	理论分析	13
3.4	实验结果分析	14
3.5	存在的问题	14

4 研究和探索的问题	15
4.1 CRC 校验能力	15
4.2 CRC 计算	16
4.3 程序设计方面的问题	16
4.3.1 协议软件的跟踪调试	16
4.3.2 get-ms 函数的实现	16
4.3.3 变长参数的调试函数	17
4.3.4 定时器处理	17
4.4 软件测试方面的问题	18
4.5 对等协议实体之间的流量控制	18
4.6 对比标准协议	18
5 实验总结和心得	19
6 源程序文件	19
6.1 累计确认的 SR 协议	19
6.1.1 h 头文件	19
6.1.2 C 源文件	20
6.2 高误码优化的 SR 协议	24
6.2.1 h 头文件	24
6.2.2 C 源文件	25
6.3 最终优化的 SR 协议	28
6.3.1 h 头文件	28
6.3.2 C 源文件	29
6.4 gobackn 协议	34
6.4.1 h 头文件	34
6.4.2 C 源文件	34

1 实验内容和实验环境描述

1.1 实验任务和实验内容

利用所学数据链路层原理,自己设计一个滑动窗口协议,在仿真环境下编程实现有噪音信道环境下两站点之间无差错双工通信。信道模型为 8000bps 全双工卫星信道,信道传播时延 270 毫秒,信道误码率为 10^{-5} ,信道提供字节流传输服务,网络层分组长度固定为 256 字节。

通过该实验,进一步巩固和深刻理解数据链路层误码检测的 CRC 校验技术,以及滑动窗口的工作机理。滑动窗口机制的两个主要目标:(1) 实现有噪音信道环境下的无差错传输;(2) 充分利用传输信道的带宽。在程序能够稳定运行并成功实现第一个目标之后,运行程序并检查在信道没有误码和存在误码两种情况下的信道利用率。为实现第二个目标,提高滑动窗口协议信道利用率,需要根据信道实际情况合理地给协议配置工作参数,包括滑动窗口的大小和重传定时器时限以及 ACK 搭载定时器的时限。这些参数的设计,需要充分理解滑动窗口协议的工作原理并利用所学的理论知识,经过认真的推算,计算出最优取值,并通过程序的运行进行验证。

本次实验,我设计实现的滑动窗口协议有

1.SR 选择重传协议

2.SR 协议在高误码率时的优化

3.SR 协议的进一步优化,三个版本均使用了 NAK 通知机制

4.捎带确认(无 ACK 通知)与 NAK 通知的 GBN 协议(由于其整体性能低于 SR 协议且思路整体一致,不做重点分析)

1.2 实验环境

WSL2 中的默认 Ubuntu 发行版 Linux

使用 neovim 编辑器开发,Makefile 自动化编译构建工程

2 软件设计

2.1 带 NAK 帧通知的 SR 协议及优化

2.1.1 数据结构

数据帧格式设计

Kind(1)	ACK(1)	SEQ(1)	DATA(256)	CRC(4)
---------	--------	--------	-----------	--------

ACK 和 NAK 帧格式设计

Kind(1)	ACK(1)	CRC(4)
---------	--------	--------

```
typedef struct FRAME {
    unsigned char kind;
    unsigned char ack;
    unsigned char seq;
    unsigned char data[PKT_LEN];
    unsigned int padding; // used for CRC detecting error
} frame;
```

其中, kind 一字节, ack 一字节, seq 一字节, data240-256 字节, crc 四字节。

一个数据帧的长度并不是可变的。通过查阅指导书, 发现网络层中的包长为固定的 256 字节, 因此一个数据帧的长度为固定的 $256+1+1+1+4=263$ 字节。

Kind 表示帧种类。

Ack 表示发送该帧的站点的上一接收的帧序列号。

Seq 表示帧序列号。

Crc 用于差错检验

将 ACK 字段放在 Seq 字段前, 便于 ACK 帧的统一, 可节省空间

初代协议中定义的全局变量

```
14 static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
13 static bool phl_ready = false;
12 static bool no_nak = true;
11 static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
10 static unsigned char frame_nr = 0; //发送帧编号, 发送窗口上界
9 static unsigned char ack_expected = 0; //发送窗口下界
8 static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
7 static unsigned char frame_expected = 0;
6 //接收窗口下界
5 //首个未收到的数据单元序号, 可能在接收窗口内, 也可能在外
4 static bool arrived[NR_BUFS]; //从全序列号到窗口大小的半映射 bitmap
3 //接收窗口的确认以便重发 NAK
2 static bool be_acked[NR_BUFS]; //发送窗口内被确认的
1 static void put_frame(unsigned char *frame, int len);
```

初代协议中定义的全局宏

```
36 //进行判断而不进行赋值
35 #define inc(k) (k == MAX_SEQ ? 0 : k + 1)
34 #define dec(k) (k == 0 ? MAX_SEQ : k - 1)
33 #define DATA_TIMER (1500)
32 #define ACK_TIMER (400)
31
30 /* FRAME kind */
29 #define FRAME_DATA (1)
28 #define FRAME_ACK (2)
27 #define FRAME_NAK (3)
26
25 #define MAX_SEQ (31)
24 #define NR_BUFS ((MAX_SEQ + 1) / 2)
23
```

优化协议中定义的全局变量

```
static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
static bool phl_ready = false;
static bool no_nak = true;
static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
static unsigned char frame_nr = 0; //发送帧编号, 发送窗口上界,
//注意: 指的是还未发送的最优先编号
static unsigned char ack_expected = 0; //发送窗口下界
static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
static unsigned char frame_expected = 0;
//接收窗口下界
//首个未收到的数据单元序号, 可能在接收窗口内, 也可能在外
static bool arrived[MAX_SEQ + 1]; //从全序列号到窗口大小的映射 bitmap
//但在这里有冲突避免接收窗口的确认以便重发 NAK
static bool be_acked[MAX_SEQ + 1]; //发送窗口内被确认的
```

优化协议中定义的全局宏

```
31 #define inc(k) (k == MAX_SEQ ? 0 : k + 1)
30 #define dec(k) (k == 0 ? MAX_SEQ : k - 1)
29 //重传时间应该从1500放宽至2000
28 #define DATA_TIMER (2000)
27 #define ACK_TIMER (400)
26
25 /* FRAME kind */
24 #define FRAME_DATA (1)
23 #define FRAME_ACK (2)
22 #define FRAME_NAK (3)
21 #define FRAME_SEQ_ACK (4)
20 #define MAX_SEQ (63)
19 #define NR_BUFS ((MAX_SEQ + 1) / 2)
```

2.1.2 模块结构

2.1.2.1 初代协议

between 函数用于判断循环意义下参数 b 是否处在 a 与 c 之间, 由于教材上给出的 between 函数是以左闭右开区间进行判断边界范围, 在协议中可能会在理解上造成歧义, 故在后续协议实践中略作修改, 改为了两侧封闭区间的判断.

putframe 函数将帧进行 CRC 校验并发送到物理层, 参数 frame 为帧的首地址, 参数 len 表示不含校验和的总长度

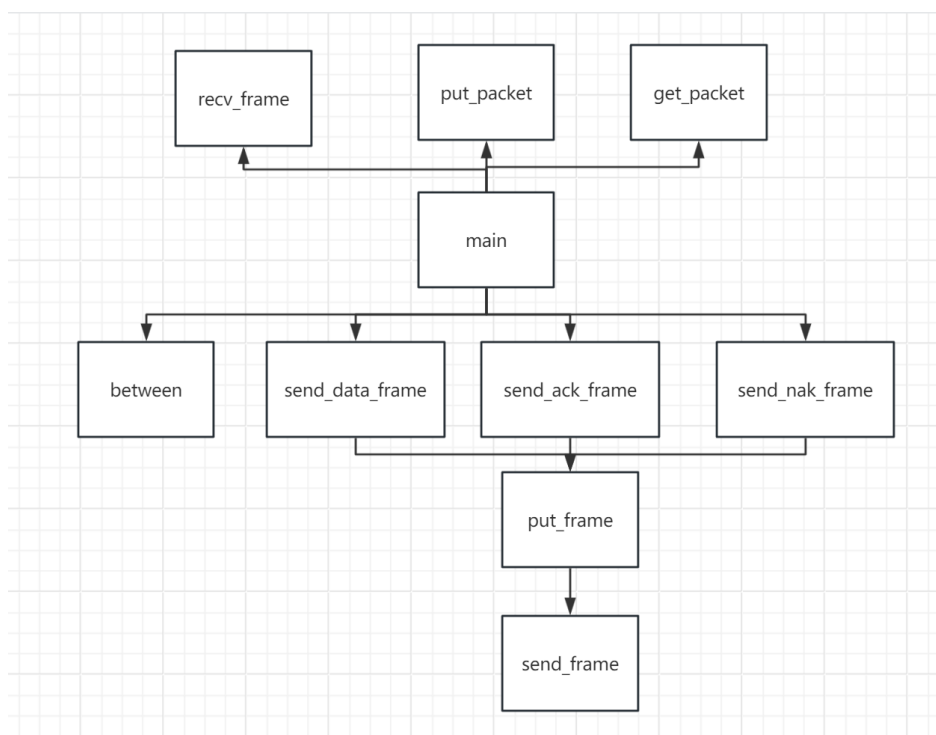
senddataframe 函数发送数据帧, 参数 seq 为将要发送的帧序号, 函数会从发送窗口缓冲区中的对应位置取得 data 数据字段.

sendackframe 函数发送 ACK 帧, 采取的是累计确认的方式.

sendnakframe 函数发送 NAK 帧, 保证单工链路上至多只有一个有效的 NAK 帧被传送

```
1 static bool between(unsigned char a, unsigned char b, unsigned char c);
2
3 static void put_frame(unsigned char *frame, int len);
4
5 static void send_data_frame(unsigned char seq);
6
7 static void send_ack_frame(void);
8
9 static void send_nak_frame(void);
```

模块之间的基本调用关系图为



后续版本的模块调用关系图大致上与本图一致, 故不再重复

2.1.2.2 优化协议

实现优化的一个重要思路就是将累计确认机制改变为分别确认 ACK, 但是由于开发库的协议限制: 在 ACK-TIMEOUT 事件发生时, 无法确认具体传递的帧序号, 故而只能放弃 ACK-TIMER 的捎带确认机制, 转而发送对应的 ACK 帧. 具体实现中, 这样的协议在高数据传输的信道上工作效率下降的比例并不大, 可以考虑先采纳这样的做法

对应修改的 ACK 帧发送模块改为

```

1 static void send_ack_frame(unsigned char ack_num) {
2     frame f;
3     f.kind = FRAME_ACK;
4     f.ack = ack_num;
5     dbg_frame("Send ACK %d\n", f.ack);
6
7     put_frame((unsigned char *)&f, 2);
8 }

```

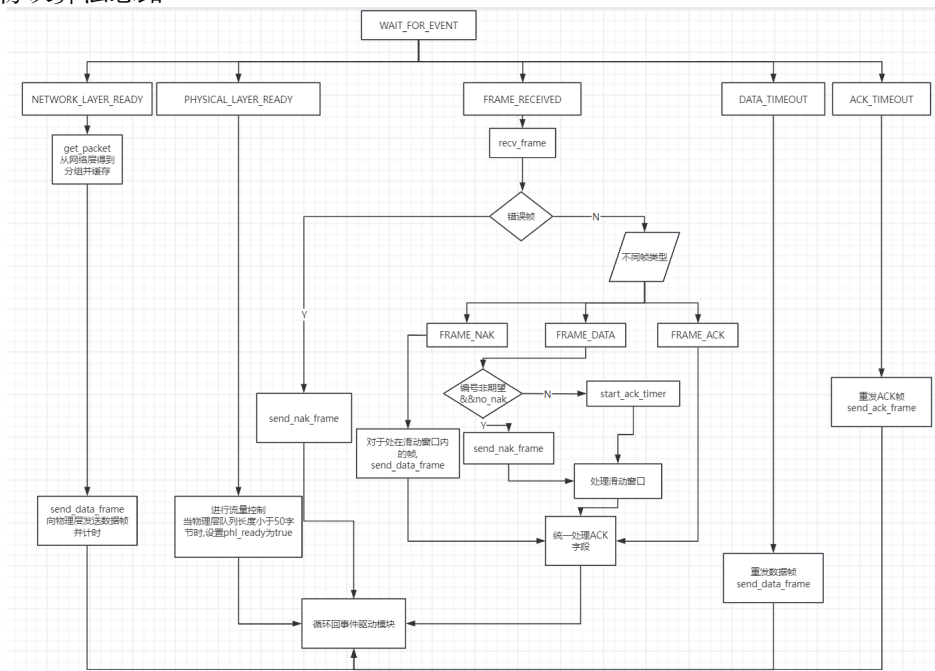
其中参数 ack-num 表示需要被确认的数据帧序号
其余模块不发生改变

2.1.3 算法流程

程序整体采用事件循环的形式, 但不是真正的事件驱动型, 因为程序采用的是单线程循环模式, 并不会出现诸如某个帧错序到达导致其被缓冲进接收窗口缓冲区时, 其他的帧确认它已经到达导致出现矛盾的多线程故障, 整体逻辑也很清晰

后续的优化 SR 协议整体框架也没有发生改变, 只是在具体的细节上有所变动: 如增加了确认接受窗口缓冲区中到达的 arrived 数组实现 ACK 帧的分别确认, 故不再重复, 这里给出基本的

SR 协议算法思路



2.2 分别确认和累计确认相结合的 SR 协议

2.2.1 数据结构

数据帧格式设计

Kind(1)	ACK(1)	SEQ(1)	DATA(256)	CRC(4)
---------	--------	--------	-----------	--------

ACK 和 NAK 和 SEQ-ACK 帧格式设计

Kind(1)	ACK(1)	CRC(4)
---------	--------	--------

```
typedef struct FRAME {
    unsigned char kind;
    unsigned char ack;
    unsigned char seq;
    unsigned char data[PKT_LEN];
    unsigned int padding; // used for CRC detecting error
} frame;
```

基本帧结构同上, 但加入了用于分别确认的 SEQ-ACK 帧

协议整体采用 ACK 累计确认的方式滑动窗口, 但也通过 SEQ-ACK 帧的分别确认机制提高协议效率

具体实现思路见后文

定义的全局变量与初次优化的变量没有区别

```
static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
static bool phl_ready = false;
static bool no_nak = true;
static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
static unsigned char frame_nr = 0; //发送帧编号, 发送窗口上界,
//注意: 指的是还未发送的最优先编号
static unsigned char ack_expected = 0; //发送窗口下界
static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
static unsigned char frame_expected = 0;
//接收窗口下界
//首个未收到的数据单元序号, 可能在接收窗口内, 也可能在外
static bool arrived[MAX_SEQ + 1]; //从全序列号到窗口大小的映射·bitmap
//但在这里有冲突避免·接收窗口的确认以便重发 NAK
static bool be_acked[MAX_SEQ + 1]; //发送窗口内被确认的
```

定义的全局的宏中新增了 FRAME-SEQ-ACK 帧类型

```
#define inc(k) (k == MAX_SEQ ? 0 : k + 1)
#define dec(k) (k == 0 ? MAX_SEQ : k - 1)
//重传时间应该从1500放宽至2000
#define DATA_TIMER (2000)
#define ACK_TIMER (400)

/* FRAME kind */
#define FRAME_DATA (1)
#define FRAME_ACK (2)
#define FRAME_NAK (3)
#define FRAME_SEQ_ACK (4)
#define MAX_SEQ (63)
#define NR_BUFS ((MAX_SEQ + 1) / 2)
```

2.2.2 模块结构

本协议中的 ACK 帧仍然遵循累计确认的滑动窗口逻辑, 但在接收到错序到达的数据帧 (包括不在接收的滑动窗口内部的数据帧) 时, 选择发送 SEQ-ACK 帧

```
1 // 累计确认ack和分别确认seq-ack
2 static void send_ack_frame(void) {
3     frame f;
4     f.kind = FRAME_ACK;
5
6     f.ack = dec(frame_expected);
7     dbg_frame("Send ACK %d\n", f.ack);
8
9     put_frame((unsigned char *)&f, 2);
10    //对于主动发送ack的类型处理
11    // stop_ack_timer();
12 }
13 static void send_seq_ack_frame(unsigned char ack_num) {
14     frame f;
15     f.kind = FRAME_SEQ_ACK;
16     f.ack = ack_num;
17     dbg_frame("Send SEQ ACK %d\n", f.ack);
18     put_frame((unsigned char *)&f, 2);
19 }
```

其中在发送 seq-ack 帧模块的参数 ack-num 作用同原优化版本的 ack-num

2.2.3 算法流程

总体思路是初代版本的 SR 协议和优化版本的 SR 协议的结合, 故思路基本同上

3 实验结果分析

3.1 程序正确性和健壮性

程序能正确按协议流程运行 30 分钟以上, 具备健壮性.

```
2242.289 ..... 8297 packets received, 7583 bps, 94.78%, Err 176 (9.8e-06)
2244.337 ..... 8305 packets received, 7583 bps, 94.78%, Err 176 (9.8e-06)
2246.436 ..... 8313 packets received, 7583 bps, 94.79%, Err 176 (9.8e-06)
2248.562 ..... 8321 packets received, 7583 bps, 94.79%, Err 176 (9.8e-06)
2250.671 ..... 8329 packets received, 7583 bps, 94.79%, Err 176 (9.8e-06)
2252.781 ..... 8337 packets received, 7583 bps, 94.79%, Err 177 (9.8e-06)
2254.877 ..... 8341 packets received, 7580 bps, 94.75%, Err 178 (9.9e-06)
2256.985 ..... 8352 packets received, 7583 bps, 94.79%, Err 179 (9.9e-06)
2259.114 ..... 8359 packets received, 7582 bps, 94.78%, Err 179 (9.9e-06)
2261.228 ..... 8367 packets received, 7582 bps, 94.78%, Err 179 (9.9e-06)
2264.376 ..... 8375 packets received, 7579 bps, 94.74%, Err 180 (9.9e-06)
2266.501 ..... 8386 packets received, 7582 bps, 94.77%, Err 180 (9.9e-06)
2268.621 ..... 8394 packets received, 7582 bps, 94.77%, Err 180 (9.9e-06)
2270.715 ..... 8402 packets received, 7582 bps, 94.78%, Err 180 (9.9e-06)
2272.873 ..... 8418 packets received, 7582 bps, 94.78%, Err 180 (9.9e-06)
2274.982 ..... 8418 packets received, 7582 bps, 94.78%, Err 180 (9.9e-06)
2277.095 ..... 8426 packets received, 7582 bps, 94.78%, Err 180 (9.9e-06)
2279.211 ..... 8434 packets received, 7583 bps, 94.78%, Err 180 (9.9e-06)
2282.181 ..... 8441 packets received, 7579 bps, 94.74%, Err 181 (9.9e-06)
2243.116 ..... 4511 packets received, 4121 bps, 51.51%, Err 101 (1.0e-05)
2245.248 ..... 4519 packets received, 4124 bps, 51.55%, Err 101 (1.0e-05)
2247.361 ..... 4527 packets received, 4120 bps, 51.60%, Err 101 (1.0e-05)
2249.471 ..... 4535 packets received, 4131 bps, 51.64%, Err 101 (1.0e-05)
2251.582 ..... 4543 packets received, 4135 bps, 51.68%, Err 101 (1.0e-05)
2253.692 ..... 4551 packets received, 4130 bps, 51.73%, Err 101 (1.0e-05)
2255.832 ..... 4559 packets received, 4141 bps, 51.77%, Err 101 (1.0e-05)
2257.958 ..... 4567 packets received, 4145 bps, 51.81%, Err 101 (1.0e-05)
2260.087 ..... 4575 packets received, 4140 bps, 51.85%, Err 101 (1.0e-05)
2262.205 ..... 4583 packets received, 4151 bps, 51.89%, Err 101 (1.0e-05)
2264.330 ..... 4591 packets received, 4155 bps, 51.93%, Err 101 (1.0e-05)
2266.455 ..... 4599 packets received, 4158 bps, 51.98%, Err 101 (1.0e-05)
2268.559 ..... 4607 packets received, 4161 bps, 52.02%, Err 101 (1.0e-05)
2271.473 ..... 4614 packets received, 4162 bps, 52.03%, Err 103 (1.0e-05)
2273.506 ..... 4625 packets received, 4168 bps, 52.11%, Err 103 (1.0e-05)
2275.696 ..... 4633 packets received, 4172 bps, 52.15%, Err 103 (1.0e-05)
2277.889 ..... 4641 packets received, 4175 bps, 52.19%, Err 103 (1.0e-05)
2279.926 ..... 4649 packets received, 4170 bps, 52.23%, Err 103 (1.0e-05)
2282.840 ..... 4657 packets received, 4182 bps, 52.27%, Err 103 (1.0e-05)
```

3.2 协议参数的选取

3.2.1 初代参数

3.2.1.1 滑动窗口大小

数据帧从物理层发出时的长度为 263 字节

结合实际考虑, 捎待确认的时间优于 ACKTimer 超时导致的 ACK 帧确认的时间
对应搭载 ACK 时的滑动窗口大小计算过程

$$\begin{cases} t_{prop} = 270ms & (1) \\ t_{send} = \frac{263 \times 8}{8000} \times 1000 = 263ms & (2) \\ \frac{W \times t_{send}}{2 \times (t_{send} + t_{prop})} \geq 1 & (3) \end{cases}$$

得到滑动窗口大小 $W \geq 4.05$ 由于实际情况 ACK 的发送有延时, 故可将窗口设置为 8
对应的最大序列号则为 15

3.2.1.2 两种定时器超时参数

对于 ACK 计时器, 在有数据帧被不断发送的时候选择通过捎带传输

在线路空闲的时候通过发送 ACK 帧进行传输

考虑到对于指定站点, 发送数据帧之间的间隔至少为 t_{send}

(在本实验中帧边界保留为 1ms 用于帧与帧之间的区分)

故可设置 ACKtimer 的大小略大于 264ms, 初代设置为 0.4s

由于 ACK 的捎带确认机制, 实际情况下接收方会等待一段时间再发送 ACK 帧, 记作

$t_{wait-data}$

ACKTimer 和 DataTimer 之间的关系是

$$t_{wait-data} + 2 \times t_{prop} + 2 \times t_{send} \leq t_{wait-ack}$$

数据超时参数:

搭载 ACK 时, 数据帧发送到接受 ACK 的时间应为 $2 \times 263 + 2 \times 270 + 0.4 = 1466ms$

设置对应数据超时通知为 1.5s

不搭载 ACK 时, 理想数据帧发送到接受 ACK 的时间应为 $263 + 2 \times 270 = 803ms$

设置对应数据超时通知为 1s

3.2.2 参数改进

3.2.2.1 基本模型

ACK 捎带确认机制中包含两种情形: ACK 单独发送或捎带发送, 能保证 $t_{wait-data} \leq ACKTimer$ 为提供信道利用率, 在滑动窗口大小已定的前提下, 满足以下条件

$$\begin{cases} t_{wait-data} + 2 \times t_{prop} + 2 \times t_{send} \leq t_{wait-ack} & (4) \\ t_{send} = \frac{263 \times 8}{8000} \times 1000 = 263ms & (5) \\ \frac{W \times t_{send}}{2 \times (t_{send} + t_{prop})} \geq 1 & (6) \end{cases}$$

3.2.2.2 解决无意义的重传

在默认误码率的 flood 模式下, A,B 站点分组的线路利用率只有 76.57%, 远远低于理论利用率

通过 debug 查看日志, 发现其中的一个问题

下面为对应两个站点的调试信息

```

031.309 Send DATA 30 25, ID 10094
031.567 Recv DATA 26 23, ID 20090
031.567
031.567 Want to put 26 to the network and ready to put frame which ID is 20090 to the network layer
031.567 when the window size is [ 26 , 9 ]
031.567 .... 91 packets received, 6108 bps, 76.35%, Err 2 (8.1e-06)
031.765 — DATA 25 timeout
031.765 Send DATA 25 26, ID 10089
031.841 Recv DATA 27 23, ID 20091
031.841
031.841 Want to put 27 to the network and ready to put frame which ID is 20091 to the network layer
031.841 when the window size is [ 27 , 10 ]
031.841 — DATA 26 timeout
032.024 Send DATA 26 27, ID 10090
032.100 Recv DATA 28 23, ID 20092
032.100
032.100 Want to put 28 to the network and ready to put frame which ID is 20092 to the network layer
032.100 when the window size is [ 28 , 11 ]
032.100 Recv DATA 25 23, ID 20089
032.348 Send NAK 28
032.348 Send DATA 31 28, ID 10095
032.562 — DATA 27 timeout
032.562 Send DATA 27 28, ID 10091
032.607 Recv DATA 29 23, ID 20093
032.607
032.607 Want to put 29 to the network and ready to put frame which ID is 20093 to the network layer
032.607 when the window size is [ 29 , 12 ]
032.607 Recv DATA 30 23, ID 20094
032.883
032.883 Want to put 30 to the network and ready to put frame which ID is 20094 to the network layer
032.883 when the window size is [ 30 , 13 ]
032.883 Send DATA 0 30, ID 10096
033.080 — DATA 28 timeout
033.080 Send DATA 28 30, ID 10092
033.096 — DATA 24 timeout
033.096 Send DATA 24 30, ID 10088
033.096 Recv DATA 29 23, ID 20093
032.054 Send DATA 29 23, ID 20093
032.130 Recv DATA 30 25, ID 10094
032.317 Send DATA 30 23, ID 20094
032.378 Recv DATA 25 26, ID 10089
032.576 Send DATA 31 23, ID 20095
032.653 Recv DATA 26 27, ID 10090
032.653 Recv NAK 28
032.653 Send DATA 29 23, ID 20093
032.928 Recv DATA 31 28, ID 10095
033.110 Send DATA 0 23, ID 20096
033.186 Recv DATA 27 28, ID 10091
033.370 Send DATA 1 23, ID 20097
033.447 Recv DATA 0 30, ID 10096
033.629 Send DATA 2 23, ID 20098
033.720 Recv DATA 28 30, ID 10092
033.903 Send DATA 3 23, ID 20099
033.979 Recv DATA 24 30, ID 10088
033.979
033.979 Want to put 24 to the network and ready to put frame which ID is 10088 to the network layer
033.979 when the window size is [ 24 , 7 ]
033.979 .... 89 packets received, 5539 bps, 69.24%, Err 4 (1.5e-05)
033.979
033.979 Want to put 25 to the network and ready to put frame which ID is 10089 to the network layer
033.979 when the window size is [ 25 , 8 ]
033.979
033.979 Want to put 26 to the network and ready to put frame which ID is 10090 to the network layer
033.979 when the window size is [ 26 , 9 ]
033.979
033.979 Want to put 27 to the network and ready to put frame which ID is 10091 to the network layer
033.979 when the window size is [ 27 , 10 ]
033.979
033.979 Want to put 28 to the network and ready to put frame which ID is 10092 to the network layer
033.979 when the window size is [ 28 , 11 ]
033.979
033.979 Want to put 29 to the network and ready to put frame which ID is 10093 to the network layer
033.979 when the window size is [ 29 , 12 ]
033.979
033.979 Want to put 30 to the network and ready to put frame which ID is 10094 to the network layer
033.979 when the window size is [ 30 , 13 ]
033.979
033.979 Want to put 31 to the network and ready to put frame which ID is 10095 to the network layer
033.979 when the window size is [ 31 , 14 ]
033.979
033.979 Want to put 0 to the network and ready to put frame which ID is 10096 to the network layer
033.979 when the window size is [ 0 , 15 ]
033.979 Send DATA 0 30, ID 10096

```

在发送连续的帧时，如 24-30 号数据帧的连续发送，如果出现 24 号帧发生错误，但其他帧正常到达

由于初代 SR 协议设计的过程是累计确认，发送方无法确定后续帧的到达情况

只能根据返回的 NAK 帧进行对应数据帧的重传，此时由于发送计时器的时限设置的并不大，一些数据帧的计时器开始超时，被迫进行重传

但接收方并不需要这些无意义的冗余数据帧, 显然它们占据了大量线路带宽
甚至如果数据帧的错误重传时间在接收方收到正确的数据帧重传后
此时的滑动窗口不再包含该冗余数据帧, 线路上又会发送多余的 NAK 以及数据帧, 周而复始...

解决方法:

1. 为避免无谓的冗余数据帧被反复发送, 将 NAK 的发送逻辑改为

线路上可以发送 $NAK \cap (\text{收到的数据帧序号} \neq \text{期望的数据帧序号}) \cap \text{收到的数据帧序号在对应的接收窗口内部}$

这样避免了来自非接收窗口的迟到的数据帧的反复被传输的问题

2. 上述方法无法避免初次冗余数据帧的重传, 可以通过扩大数据帧计时器的时限

使得前方的重传帧到达接收方之后, 后方的数据帧重传计时器还未超时, 从而降低冗余数据帧的重传概率

在本程序中可以考虑首个数据帧重传的时间与下一个数据帧被接收窗口缓冲的时间基本相同

解释: 这是由于本程序中数据帧发送时间和传输时间大致相同, 通过计算对比可以得到上述结论

故再经过 $t_{send} + t_{prop} = 533\text{ms}$, 滑动窗口的位置发生改变, ACK 帧被正常发送回发送方

即在原先的最大计时器上限的基础上添加 533ms, 最终计时器为 1999ms

通过调整参数发现, 数据计时器时限过大也会导致利用率下降

显然这是因为如果在单一数据帧错误的情况下, 滑动窗口始终得不到滑动, 无法向网络层传递数据包

最终设定计时器为 2s

3. 相应地, 在调整数据重传计时器后, 发现此时的滑动窗口大小限制了数据包的发送

有时, 整个滑动窗口被后续的数据包填满, 但由于初始帧出错, 无法得到发送

出现如下情形

```

32 051.240 when the window size is [ 28 , 11 ]
31 051.240 .... 125 packets received, 5027 bps, 62.84%, Err 41 (1.1e-04)
30 051.240
29 051.240 Want to put 29 to the network and ready to put frame which ID is 20125 to the network layer
28 051.240 when the window size is [ 29 , 12 ]
27 051.240
26 051.240 Want to put 30 to the network and ready to put frame which ID is 20126 to the network layer
25 051.240 when the window size is [ 30 , 13 ]
24 051.240
23 051.240 Want to put 31 to the network and ready to put frame which ID is 20127 to the network layer
22 051.240 when the window size is [ 31 , 14 ]
21 051.240
20 051.240 Want to put 0 to the network and ready to put frame which ID is 20128 to the network layer
19 051.240 when the window size is [ 0 , 15 ]
18 051.240
17 051.240 Want to put 1 to the network and ready to put frame which ID is 20129 to the network layer
16 051.240 when the window size is [ 1 , 16 ]
15 051.240
14 051.240 Want to put 2 to the network and ready to put frame which ID is 20130 to the network layer
13 051.240 when the window size is [ 2 , 17 ]
12 051.240
11 051.240 Want to put 3 to the network and ready to put frame which ID is 20131 to the network layer
10 051.240 when the window size is [ 3 , 18 ]
9 051.240
8 051.240 Want to put 4 to the network and ready to put frame which ID is 20132 to the network layer
7 051.240 when the window size is [ 4 , 19 ]
6 051.240
5 051.240 Want to put 5 to the network and ready to put frame which ID is 20133 to the network layer
4 051.240 when the window size is [ 5 , 20 ]
3 051.240
2 051.240 Want to put 6 to the network and ready to put frame which ID is 20134 to the network layer
1 051.240 when the window size is [ 6 , 21 ]
1366 051.240
1 051.240 Want to put 7 to the network and ready to put frame which ID is 20135 to the network layer
2 051.240 when the window size is [ 7 , 22 ]
3 051.240
4 051.240 Want to put 8 to the network and ready to put frame which ID is 20136 to the network layer
5 051.240 when the window size is [ 8 , 23 ]
6 051.240
7 051.240 Want to put 9 to the network and ready to put frame which ID is 20137 to the network layer
8 051.240 when the window size is [ 9 , 24 ]
9 051.240
10 051.240 Want to put 10 to the network and ready to put frame which ID is 20138 to the network layer
11 051.240 when the window size is [ 10 , 25 ]
12 051.240
13 051.240 Want to put 11 to the network and ready to put frame which ID is 20139 to the network layer
14 051.240 when the window size is [ 11 , 26 ]

```

故将滑动窗口大小翻倍, 设置为 32

那么滑动窗口的大小是不是越大越好呢?

显然不是, 这与网络传输能力 (带宽时延积) 的大小有关

参考文章来源 [1]

3.2.3 最终优化

由于针对分别确认的 SR 协议的效率在一定程度上还有较大改进空间, 故设计分别确认和累计确认相结合的 SR 协议

相应的参数理论来源同上

具体参数: 滑动窗口大小为 32, 数据重传计时器大小为 2s, ACK 发送计时器大小为 400ms

3.3 理论分析

假设信道误码率为 e

对于每一个长度为 263 字节的数据帧, 其一次成功发送的概率为

$$P = (1 - e)^{263 \times 8} = (1 - e)^{2104}$$

假设一个帧传输成功需要的次数作为随机变量 X , 有

$$E[X] = \sum_{i=0}^{\infty} i \times (1-P)^{i-1} \times P = \frac{1}{P}$$

所以当信道无误码时, 理论最高线路利用率为

$$\frac{256}{263} = 97.3\%$$

信道误码率为 $1e-5$ 时, 理论最高线路利用率为

$$\frac{256}{263 \times X} = 95.31\%$$

信道误码率为 $1e-4$ 时, 理论最高线路利用率为

$$\frac{256}{263 \times X} = 78.86\%$$

实际情况还需要考虑 ACK 和 NAK 占据的带宽会导致实际线路利用率进一步下降并且捎带 ACK 或者单独发送 ACK 和 NAK 时也可能出错

3.4 实验结果分析

实验最终性能表如图所示

序号	命令选项	说明	运行时间(秒)	Selective piggybacking 初代 线路利用率(%)		Selective Noncumulative confirmation + Cumulative confirmation 线路利用率(%)	
				A	B	A	B
1	--utopia	无误码信道数据传输	1213	51.95%	96.97%	58.71%	96.97%
2	无	站点 A 分组层平缓方式发出数据, 站点 B 周期性交替“发送 100 秒, 停发 100 秒”	2300	47.86%	86.93%	52.57%	94.79%
3	--flood --utopia	无误码信道, 站点 A 和站点 B 的分组层都洪水式产生分组	1058	96.97%	96.97%	96.97%	96.97%
4	--flood	站点 A/B 的分组层都洪水式产生分组	1168	76.51%	76.57%	94.30%	94.80%
5	--flood --ber=1e-4	站点 A/B 的分组层都洪水式产生分组, 线路误码率设为 10^{-4}	1899	56.62%	57.88%	77.62%	76.23%

本协议最终较好地实现了在有误码的信道上的高效率无差错传输功能

3.5 存在的问题

在理论分析时, 本协议直接使用开发库提供的成帧完毕的数据进行分析, 实际实现中, 需要结合不同的成帧方式以分析效率.

另外, 协议参数的选择基本依照高数据传输率的情形进行分析, 当两个站点通信情况发生变化时, 这种静态的固定参数不能较好的适配复杂的网络环境, 故而后续的改进思路在于自适应的流量控制.

同时, 对于高误码率的情况, 每个帧基本都有多次重发的可能性, 这种情形可以将检错码替换为纠错码

本程序中 main 函数中的模块还可以继续划分为子模块以降低理解成本

滑动窗口大小的最优选择还无法确定, 但一定小于 129, 这是由于开发库限制了定时器大小为 129 个

4 研究和探索的问题

4.1 CRC 校验能力

CRC 校验的作用是用于对抗传输信道中随机错误的, 也即为了检测出在信号传输系统中由于硬件问题导致的错误, 比如位翻转、截断等.

其基本实现思路为

Code polynomials can be formed by simply multiplying any polynomial of degree less than k by $P(X)$. The following method has the advantage, however, that it results in a code polynomial in which the high-order coefficients are message symbols and the low-order coefficients are check symbols. To encode a message polynomial $G(X)$, we divide $X^{n-k}G(X)$ by $P(X)$ and then add the remainder $R(X)$ resulting from this division to $X^{n-k}G(X)$ to form the code polynomial:

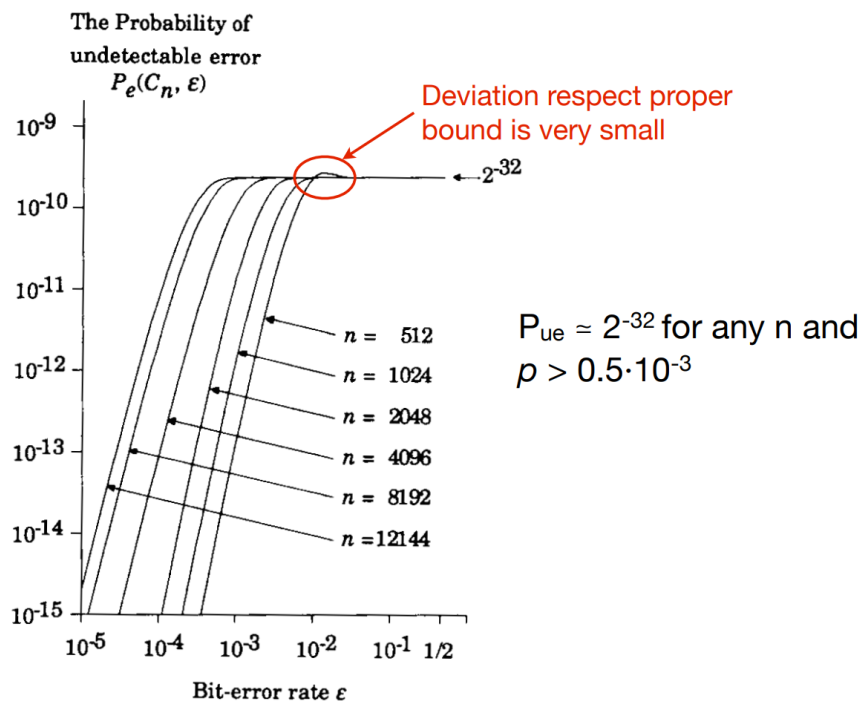
$$X^{n-k}G(X) = Q(X)P(X) + R(X),$$

where $Q(X)$ is the quotient and $R(X)$ the remainder resulting from dividing $X^{n-k}G(X)$ by $P(X)$. Since in modulo two arithmetic, addition and subtraction are the same,

$$F(X) = X^{n-k}G(X) + R(X) = Q(X)P(X),$$

which is a multiple of $P(X)$ and, therefore, a code polynomial. Furthermore, $R(X)$ has degree less than $n-k$, and $X^{n-k}G(X)$ has zero coefficients in the $n-k$ low-order terms. Thus the k highest-order coefficients of $F(X)$ are the same as the coefficients of $G(X)$, which are the message symbols. The low order $n-k$ coefficients of $F(X)$ are the coefficients of $R(X)$, and these are the check symbols.

CRC32 本身可以检验所有一位, 两位, 奇数位错误, 和长度小于等于 32 位的突发错误
根据相关研究



对于本信道中 263 字节, $n=2104$ 位的消息来说, 在误码率为 $1e-5$ 的信道中, 出错率小于 $1e-15$
 该用户信道使用率为 50%
 故发生一次分组层误码的年数为

$$\frac{10^{15} \text{ times } 263}{50\% \times 1000 \times 60 \times 60 \times 24 \times 365} = 1.668 \times 10^7$$

这是非常保守的估计, 实际效果更加优秀

4.2 CRC 计算

查表法和逐位计算本质上等价, 这部分参考 [2]

4.3 程序设计方面的问题

4.3.1 协议软件的跟踪调试

软件跟踪调试系统指的是运行期对协议动作进行跟踪和观察分析以确定协议逻辑是否正确, 方便开发者进行调试分析。我的程序中使用到了 `lprintf`, `dbg-frame`, `dbg-event` 系列调试函数, 较好的帮助我进行开发。

4.3.2 get-ms 函数的实现

开发库中的 `get-ms` 函数通过操作系统中的一些被弃用的结构体进行从程序开启的时间戳到当前的时间大小的计算。

可以通过一个不受系统时间影响的一个单调时钟作为参考, 同时将精准度设置到纳米级别后, 再进行转换

```
static struct timespec base_ts;

// 初始化基准时间
void init_timer() { clock_gettime(CLOCK_MONOTONIC_RAW, &base_ts); }

// 获取相对于基准时间的毫秒数
unsigned int get_ms(void) {
    struct timespec curr_ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &curr_ts);

    uint64_t base_ms = base_ts.tv_sec * 1000 + base_ts.tv_nsec / 1000000;
    uint64_t curr_ms = curr_ts.tv_sec * 1000 + curr_ts.tv_nsec / 1000000;

    return (unsigned int)curr_ms - base_ms;
}
```

然后在 protocol-init 函数中加入 init-timer 初始化基准时间。

在实现 get-ms 函数时, 我更好的理解了指导书中所说的”默认情况下, 协议程序会持续运行 25 天”的含义, 源程序中默认给出的 mode-life 时间为 0x7ffff00, 显然表示的是 ms 故而进行运算

$$0x7FFFFFF00 \div 1000 \div 60 \div 60 \div 24 \approx 24.85(days)$$

4.3.3 变长参数的调试函数

lprintf 和 dbg-event 系列函数都采用了变长参数的方式设计, 其中... 表示对应的变长参数列表, 通过在函数定义中声明 va-list 变长参数列表, 利用 va-start 宏确认可变列表参数位置, 通过 lprintf 函数中对输入字符串进行词法分析, 通过状态转移和 va-arg 宏对应格式化输出, 最后通过 va-end 宏结束可变参数列表的访问

同时将调试信息重定向入 logfile 的过程也是 lprintf 实现的

4.3.4 定时器处理

定时器为何如此设计?

对于 start-timer 来说, 系统在把截止到调用函数 start-timer(). 它的意义在于只有当真正的数据被发送的时候, 定时器才开始计时, 比如当数据帧在物理层队列堆积较大时, 不能因为发送方自身发送数据的时延长而选择错误地进行重发。

start-timer() 函数的机制是在定时器未超时之前直接对同一个编号的定时器执行 start-timer() 调用, 将按照新的时间设置产生超时事件. 这种情形是用于处理当接收方返回 NAK 帧通知时, 发送方需要重新发送相同的帧, 此时是以当前的帧作为计时标准, 而不应该受到前述帧的干扰

对于搭载 ACK 机制的 start-ack-timer 函数来说, 定时器的启动时刻是当前时刻, 这是因为准备捎带确认的等待数据帧搭载的容忍时间是从当前开始计算的, 如果在指定的 ACK-TIMER 时间范围外就立即单独发送 ACK 帧进行确认。

start-ack-timer() 的机制是在先前启动的定时器未超时之前重新执行 start-ack-timer() 调用, 定时器将依然按照先前的时间设置产生超时事件 ACK-TIMEOUT. 在本程序中滑动窗口变化时, 会多次调用开启 ack 定时器的函数, 我们要以最早的计时器函数作为标准。

本程序虽然表面上是事件驱动, 本质上是单线程的状态转移处理. 而在实际通信环境下, 可能出现同时处理多个帧的情形, 这时不希望由于后续到达的数据帧导致的 start-ack-timer() 的调用, 导致前述帧的响应时间无限期增长

一开始应该 enable 网络层还是 disable

4.4 软件测试方面的问题

设计这么多种测试方案的目的是为了验证程序的鲁棒性, 能够适应诸如理想信道, 高误码率信道, 高数据传输率信道, 低数据传输率信道, 通信双方数据传输状态不同下的复杂环境干扰, 做到统一的高效.

我认为当前测试方案基本完善, 如果我来开发的话, 我会进行单元测试加上日志调试分析

本次实验的事件处理如果按照参考 ppt 上的做法是单线程通信, 脱离实际背景, 如果要改进的话, 可以从 C 语言多线程的角度进行教学和实践

4.5 对等协议实体之间的流量控制

本程序在物理层和数据链路层之间, 实现了流量控制, 当物理层的缓冲队列满之后, 通知网络层停止发送分组, 降低物理层的负担

但是在面向网络层的流量控制机制中, 开发库保证了网络层可以不断接收来自数据链路层的数据并加以验证, 实际情况中网络层可能会繁忙拥挤, 可以采用以下方案

1. 由网络层反馈消息通知数据链路层发送数据的速度, 同时将多余的帧丢弃
2. 双方约定特定的传输速率并保证畅通

4.6 对比标准协议

本协议没有实现成帧, 没有实现 MAC 子层上的网段上通信, 只是简单实现了 PPP 协议, 即只实现了点对点链路通信, 无法在共享介质中使用

本协议没有实现多线程, 没有实现对网络层的流量控制, 无法自适应地调整参数等

5 实验总结和心得

本次实验的实际上机调试时间大约在 15 小时, 更多在不断优化协议和调整参数的过程中, 实验指导书的内容很多但是也都很有必要, 花费时间最多的就是撰写实验报告...

我还花了一定时间研究开发库的代码编写. 调试过程中遇到过

1. 错误地把协议假设为多线程, 进行数据帧之间的按序控制问题
2. 在分别确认和累计确认结合的过程中, 对于 ACK 字段的处理产生过问题
3. 没有正确的设定 arrived 数组的大小, 以致于一度出现数据帧抢占冲突的问题

还有一系列调试过程中的思路瓶颈, 但最后都成功通过 dbg 工具成功解决

本次实验使用的是 linux 的 make 工具, 故没有较大环境问题, 通过对开发库的进一步研究, 我对 C 语言的了解更加深入

开发库中 dbg 掩码部分出现了错误, 我进行了修正

总结本次实验, 收获良多, 不仅是软件架构思路还是计算机网络的理论学习, 都有了非常好的提升

6 源程序文件

6.1 累计确认的 SR 协议

6.1.1 h 头文件

```
1 #ifndef MYDATAINK_H_
2 #define MYDATAINK_H_
3
4 //进行调参
5 #include "protocol.h"
6
7 #include <stdbool.h>
8 // 进行判断而不进行赋值
9 #define inc(k) (k == MAX_SEQ ? 0 : k + 1)
10 #define dec(k) (k == 0 ? MAX_SEQ : k - 1)
11 #define DATA_TIMER (1500)
12 #define ACK_TIMER (400)
13
14 /* FRAME kind */
15 #define FRAME_DATA (1)
16 #define FRAME_ACK (2)
17 #define FRAME_NAK (3)
18
19 #define MAX_SEQ (31)
20 #define NR_BUFS ((MAX_SEQ + 1) / 2)
21
22 static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
23 static bool phl_ready = false;
24 static bool no_nak = true;
25 static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
26 static unsigned char frame_nr = 0; //发送帧编号,发送窗口上界
27 static unsigned char ack_expected = 0; //发送窗口下界
28 static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
```

```

29 static unsigned char frame_expected = 0;
30 //接收窗口下界
31 //首个未收到的数据单元序号, 可能在接受窗口内, 也可能在外
32 static bool arrived[NR_BUFS]; //从全序列号到窗口大小的半映射 bitmap
33 //接收窗口的确认以便重发NAK
34 static bool be_acked[NR_BUFS]; //发送窗口内被确认的
35 static void put_frame(unsigned char *frame, int len);
36 static void send_data_frame(unsigned char seq);
37
38 static void send_ack_frame(void);
39 // static void send_ack_frame(unsigned char ack_num);
40 static void send_nak_frame(void);
41
42 /*
43 DATA Frame
44 +-----+-----+-----+-----+
45 | KIND(1) | ACK(1) | SEQ(1) | DATA(240~256) | CRC(4) |
46 +-----+-----+-----+-----+
47
48 ACK Frame
49 +-----+-----+-----+
50 | KIND(1) | ACK(1) | CRC(4) |
51 +-----+-----+-----+
52
53 NAK Frame
54 +-----+-----+-----+
55 | KIND(1) | ACK(1) | CRC(4) |
56 +-----+-----+-----+
57 */
58 #endif // !MYDATALINK_H

```

6.1.2 C 源文件

```

1 #include "mydatalink.h"
2 #include "protocol.h"
3 #include <stdlib.h>
4 #include <string.h>
5 typedef struct FRAME {
6     unsigned char kind;
7     unsigned char ack;
8     unsigned char seq;
9     unsigned char data[PKT_LEN];
10    unsigned int padding; // used for CRC detecting error
11 } frame;
12
13 /* 判断b是否在边界a,c之间 */
14 static bool between(unsigned char a, unsigned char b, unsigned char c) {
15     return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
16 }
17
18 // len字段保证类型统一
19 static void put_frame(unsigned char *frame, int len) {
20     *(unsigned int *)(frame + len) = crc32(frame, len);
21     send_frame(frame, len + 4);
22     phl_ready = false;
23 }
24

```

```
25 static void send_data_frame(unsigned char seq) {
26     frame f;
27     f.kind = FRAME_DATA;
28     //指定的累计确认ack字段
29     f.ack = dec(frame_expected); // i.e. dec(FRAME_EXPECTED)
30     f.seq = seq;
31     memcpy(f.data, out_buffer[seq % NR_BUFS], PKT_LEN);
32
33     dbg_frame("Send DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
34
35     put_frame((unsigned char *)&f, 3 + PKT_LEN);
36
37     //系统支持最大计数器 0到128
38     start_timer(seq, DATA_TIMER);
39
40     //稍待确认
41     stop_ack_timer();
42 }
43
44 //累计确认ack方法
45 static void send_ack_frame(void) {
46     frame f;
47     f.kind = FRAME_ACK;
48     f.ack = dec(frame_expected);
49
50     dbg_frame("Send ACK %d\n", f.ack);
51
52     put_frame((unsigned char *)&f, 2);
53     stop_ack_timer();
54 }
55
56 //分别确认ack
57 /*
58 static void send_ack_frame(unsigned char ack_num) {
59     frame f;
60     f.kind = FRAME_ACK;
61     f.ack = ack_num;
62
63     dbg_frame("Send ACK %d\n", f.ack);
64
65     put_frame((unsigned char *)&f, 2);
66
67     stop_ack_timer();
68 }*/
69
70 static void send_nak_frame(void) {
71     frame f;
72     f.kind = FRAME_NAK;
73     f.ack = dec(frame_expected);
74     dbg_frame("Send NAK %d\n", f.ack);
75     //顺序是否有要求?
76     no_nak = false;
77     put_frame((unsigned char *)&f, 2);
78
79     stop_ack_timer();
80 }
81
82 int main(int argc, char *argv[]) {
83     memset(in_buffer, 0, sizeof in_buffer);
```

```

84  memset(out_buffer, 0, sizeof out_buffer);
85  memset(arrived, false, sizeof arrived);
86  memset(be_acked, false, sizeof be_acked);
87  int event, arg;
88  int len = 0;
89  frame f;
90  protocol_init(argc, argv);
91  lprintf("Designed by Wang Zixuan, build: " __DATE__ " " __TIME__
92         "\n");
93
94  disable_network_layer();
95  for (;;) {
96      event = wait_for_event(&arg);
97      switch (event) {
98          /* 准备数据分组 */
99      case NETWORK_LAYER_READY:
100         get_packet(out_buffer[frame_nr % NR_BUFS]);
101         send_nbuffered++;
102         send_data_frame(frame_nr);
103         // be_acked[frame_nr % NR_BUFS] = false; /* 更新发送窗口状态*/
104         // 唯一的 发送窗口的扩展
105         frame_nr = inc(frame_nr);
106         break;
107      case PHYSICAL_LAYER_READY:
108         phl_ready = true;
109         break;
110
111      case FRAME_RECEIVED:
112         //坏帧
113         len = recv_frame((unsigned char *)&f, sizeof(f));
114         if (len < 6 || crc32((unsigned char *)&f, len) != 0) {
115             dbg_event("**** Receiver Error, Bad CRC Checksum\n");
116             if (no_nak) {
117                 send_nak_frame();
118             }
119             break;
120         }
121         if (f.kind == FRAME_DATA) {
122
123             /* 先处理接收逻辑*/
124             //如果失序, 首先请求重传期望的帧
125             if (f.seq != frame_expected && no_nak) {
126                 send_nak_frame();
127                 //如果传输过程中出错, 依靠发送方的DATA_TIMER定时器进行数据重传
128             }
129             //如果发送nak无反应, 继续催促
130             //对应的是谁?
131             else {
132                 start_ack_timer(ACK_TIMER);
133             }
134
135             if (between(frame_expected, f.seq, ack_nr) &&
136                 arrived[f.seq % NR_BUFS] == false) {
137                 dbg_frame("Recv DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
138                 arrived[f.seq % NR_BUFS] = true;
139                 memcpy(in_buffer[f.seq % NR_BUFS], f.data, PKT_LEN);
140                 //开始滑动窗口
141                 while (arrived[frame_expected % NR_BUFS] == true) {
142                     no_nak = true;

```

```

143         arrived[frame_expected % NR_BUFS] = false;
144         put_packet(in_buffer[frame_expected % NR_BUFS], PKT_LEN);
145         frame_expected = inc(frame_expected);
146         ack_nr = inc(ack_nr);
147         start_ack_timer(ACK_TIMER);
148     }
149 }
150 }
151 if (f.kind == FRAME_ACK) {
152     /*
153         if (be_acked[f.ack] == false) {
154             be_acked[f.ack] = true;
155             stop_timer(f.ack);
156         }
157     */
158     dbg_frame("Recv ACK %d\n", f.ack);
159 }
160 if (f.kind == FRAME_NAK) {
161     dbg_frame("Recv NAK %d\n", f.ack);
162     if (send_nbuffered != 0 &&
163         between(ack_expected, inc(f.ack), dec(frame_nr))) {
164         send_data_frame(inc(f.ack));
165     }
166 }
167 // 对于所有帧的ack字段进行处理, 累计确认
168 while (between(ack_expected, f.ack, dec(frame_nr))) {
169     stop_timer(ack_expected);
170     ack_expected = inc(ack_expected);
171     send_nbuffered--;
172 }
173 /*
174 while (be_acked[ack_expected] == true && send_nbuffered) {
175     // 确保不被错误接受
176     be_acked[ack_expected] = false;
177     ack_expected = inc(ack_expected);
178     send_nbuffered--;
179 }*/
180 break;
181 case DATA_TIMEOUT:
182     dbg_event("—— DATA %d timeout\n", arg);
183     send_data_frame(arg);
184     break;
185
186 // 如何记录全局的ACK信号?
187 // 理论上应该对每个ack_timer记录对应编号
188
189 // 无法捎带确认时
190 case ACK_TIMEOUT:
191     dbg_event("—— ACK timeout\n");
192     send_ack_frame();
193     break;
194 }
195
196 // 发送窗口未满足且物理层已准备
197 if (send_nbuffered < NR_BUFS && phl_ready)
198     enable_network_layer();
199 else
200     disable_network_layer();
201 }

```

```

202 |
203 |     return EXIT_SUCCESS;
204 | }

```

6.2 高误码优化的 SR 协议

6.2.1 h 头文件

```

1  #ifndef __MYDATAINKBETTER_H_
2  #define __MYDATAINKBETTER_H_
3
4  #include "protocol.h"
5
6  #include <stdbool.h>
7  #define inc(k) (k == MAX_SEQ ? 0 : k + 1)
8  #define dec(k) (k == 0 ? MAX_SEQ : k - 1)
9  #define DATA_TIMER (1500)
10 // #define ACK_TIMER (400) // become useless
11
12 /* FRAME kind */
13 #define FRAME_DATA (1)
14 #define FRAME_ACK (2)
15 #define FRAME_NAK (3)
16
17 #define MAX_SEQ (31)
18 #define NR_BUFS ((MAX_SEQ + 1) / 2)
19
20 static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
21 static bool phl_ready = false;
22 static bool no_nak = true;
23 static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
24 static unsigned char frame_nr = 0; //发送帧编号,发送窗口上界,
25 //注意: 指的是还未发送的最优先编号
26 static unsigned char ack_expected = 0; //发送窗口下界
27 static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
28 static unsigned char frame_expected = 0;
29 //接收窗口下界
30 //首个未收到的数据单元序号, 可能在接受窗口内, 也可能在外
31 static bool arrived[MAX_SEQ + 1]; //从全序列号到窗口大小的映射 bitmap
32 //但在这里有冲突避免 接收窗口的确认以便重发NAK
33 static bool be_acked[MAX_SEQ + 1]; //发送窗口内被确认的
34 static void put_frame(unsigned char *frame, int len);
35 static void send_data_frame(unsigned char seq);
36
37 // static void send_ack_frame(void);
38 static void send_ack_frame(unsigned char ack_num);
39 static void send_nak_frame(void);
40
41 /*
42 DATA Frame
43 |=====|=====|=====|=====|=====|
44 | KIND(1) | ACK(1) | SEQ(1) | DATA(240~256) | CRC(4) |
45 |=====|=====|=====|=====|=====|
46
47 ACK Frame
48 |=====|=====|=====|
49 | KIND(1) | ACK(1) | CRC(4) |

```

```

50  +-----+-----+-----+
51
52  NAK Frame
53  +-----+-----+-----+
54  | KIND(1) | ACK(1) | CRC(4) |
55  +-----+-----+-----+
56  */
57  #endif // !__MYDATALINKBETTER_H

```

6.2.2 C 源文件

```

1  #include "mydatalinkBetter.h"
2  #include "protocol.h"
3  #include <asm-generic/errno.h>
4  #include <stdlib.h>
5  #include <string.h>
6  typedef struct FRAME {
7      unsigned char kind;
8      unsigned char ack;
9      unsigned char seq;
10     unsigned char data[PKT_LEN];
11     unsigned int padding; // used for CRC detecting error
12 } frame;
13
14 /* 判断b是否在边界a,c之间 */
15 static bool between(unsigned char a, unsigned char b, unsigned char c) {
16     bool ret = ((a <= b) && (b <= c)) || ((c <= a) && (a <= b)) ||
17             ((b <= c) && (c <= a));
18     /*
19     if (ret) {
20         dbg_frame("\ntry to put frame %d to the network and works\
21 when the window size is [ %d , %d ]\n",
22                 b, a, c);
23     }*/
24     return ret;
25 }
26
27 // len字段保证类型统一
28 static void put_frame(unsigned char *frame, int len) {
29     *((unsigned int *) (frame + len)) = crc32(frame, len);
30     send_frame(frame, len + 4);
31     phl_ready = false;
32 }
33
34 static void send_data_frame(unsigned char seq) {
35     frame f;
36     f.kind = FRAME_DATA;
37     // 指定的累计确认ack字段
38     f.ack = dec(frame_expected); // i.e. dec(FRAME_EXPECTED)
39     f.seq = seq;
40     memcpy(f.data, out_buffer[seq % NR_BUFS], PKT_LEN);
41
42     dbg_frame("Send DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
43
44     put_frame((unsigned char *)&f, 3 + PKT_LEN);
45
46     // 系统支持最大计数器 0到128

```

```
47 start_timer(seq, DATA_TIMER);
48
49 //不再使用稍待确认和ack_timer
50 }
51
52 //分别确认ack
53
54 static void send_ack_frame(unsigned char ack_num) {
55     frame f;
56     f.kind = FRAME_ACK;
57     f.ack = ack_num;
58     dbg_frame("Send ACK %d\n", f.ack);
59
60     put_frame((unsigned char *)&f, 2);
61 }
62
63 static void send_nak_frame(void) {
64     frame f;
65     f.kind = FRAME_NAK;
66     f.ack = dec(frame_expected);
67     dbg_frame("Send NAK %d\n", f.ack);
68     //顺序是否有要求?,应该无影响
69     no_nak = false;
70     put_frame((unsigned char *)&f, 2);
71 }
72
73 //可以将main函数中的功能进一步划分模块
74 int main(int argc, char *argv[]) {
75     memset(in_buffer, 0, sizeof in_buffer);
76     memset(out_buffer, 0, sizeof out_buffer);
77     memset(arrived, false, sizeof arrived);
78     memset(be_acked, false, sizeof be_acked);
79     // unsigned char zeroArr[PKT_LEN];
80     // memset(zeroArr, 0, sizeof zeroArr);
81     int event, arg;
82     int len = 0;
83     frame f;
84     protocol_init(argc, argv);
85     lprintf("Designed by Wang Zixuan, build: " __DATE__ " " __TIME__
86           "\n");
87
88     disable_network_layer();
89     for (;;) {
90         event = wait_for_event(&arg);
91         switch (event) {
92             /* 准备数据分组 */
93             case NETWORK_LAYER_READY:
94                 get_packet(out_buffer[frame_nr % NR_BUFS]);
95                 send_nbuffered++;
96                 send_data_frame(frame_nr);
97                 be_acked[frame_nr] = false; /* 更新发送窗口状态*/
98                 // 唯一的 发送窗口的扩展
99                 frame_nr = inc(frame_nr);
100                 break;
101             case PHYSICAL_LAYER_READY:
102                 phl_ready = true;
103                 break;
104
105             case FRAME_RECEIVED:
```

```

106 //坏帧
107 len = recv_frame((unsigned char *)&f, sizeof(f));
108 if (len < 6 || crc32((unsigned char *)&f, len) != 0) {
109     dbg_event("**** Receiver Error, Bad CRC Checksum\n");
110     if (no_nak) {
111         send_nak_frame();
112     }
113     break;
114 }
115 if (f.kind == FRAME_DATA) {
116
117     /* 先处理接收逻辑*/
118     dbg_frame("Recv DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
119     //如果失序, 首先请求重传期望的帧
120     if (f.seq != frame_expected && no_nak) {
121         send_nak_frame();
122         //如果传输过程中出错, 依靠发送方的DATA_TIMER定时器进行数据重传
123     }
124     send_ack_frame(f.seq);
125
126     if (between(frame_expected, f.seq, ack_nr) &&
127         (frame_expected != ack_nr) && arrived[f.seq] == false) {
128         //拷贝需要时间!! 事件响应过程需要时间
129         //但在这里是单线程, 处理较简单
130         if (memcpy(in_buffer[f.seq % NR_BUFS], f.data, PKT_LEN) != NULL) {
131             // len在这里固定为PKT_LEN
132             dbg_frame("\n%d frame copy to the buffer %d\n", f.seq,
133                 *(short *)in_buffer[f.seq % NR_BUFS]);
134             arrived[f.seq] = true;
135         }
136         //开始滑动窗口
137         while (arrived[frame_expected] ==
138             true &&
139                 // memcmp(in_buffer[frame_expected % NR_BUFS],
140                 // zeroArr,
141                 // PKT_LEN) != 0
142         ) {
143             no_nak = true;
144             dbg_frame("\nWant to put %d to the network and ready to put frame "
145                 "which ID is %d to the network layer\n"
146                 "when the window size is [ %d , %d ]\n ",
147                 frame_expected,
148                 *(short *)in_buffer[frame_expected % NR_BUFS],
149                 frame_expected, ack_nr);
150             put_packet(in_buffer[frame_expected % NR_BUFS], PKT_LEN);
151             memset(in_buffer[frame_expected % NR_BUFS], 0, PKT_LEN);
152             arrived[frame_expected] = false;
153             // dbg_frame("\nMake %d to be false in arrived\n",
154             //     frame_expected);
155             frame_expected = inc(frame_expected);
156             ack_nr = inc(ack_nr);
157         }
158     }
159 }
160 if (f.kind == FRAME_ACK) {
161     if (be_acked[f.ack] == false) {
162         be_acked[f.ack] = true;
163         stop_timer(f.ack);
164     }

```

```

165     dbg_frame("Recv ACK %d\n", f.ack);
166 }
167
168 //如果nak和data帧优先到达, 可以按照其中的ack字段进行对应的修改, 个人认为影响不大?
169 /*
170 if ((f.kind == FRAME_NAK || f.kind == FRAME_DATA) && send_nbuffered &&
171     between(ack_expected, f.ack, dec(frame_nr))) {
172     for (unsigned char i = ack_expected; i != inc(f.ack); i = inc(i)) {
173         if (be_acked[i] == false) {
174             be_acked[i] = true;
175             stop_timer(i);
176         }
177     }
178 }
179 */
180 if (f.kind == FRAME_NAK) {
181     dbg_frame("Recv NAK %d\n", f.ack);
182     if (send_nbuffered != 0 &&
183         between(ack_expected, inc(f.ack), dec(frame_nr))) {
184         send_data_frame(inc(f.ack));
185     }
186 }
187 while (be_acked[ack_expected] == true && send_nbuffered){
188     // 确保不被错误接受
189     be_acked[ack_expected] = false;
190     ack_expected = inc(ack_expected);
191     send_nbuffered--;
192 }
193 break;
194 case DATA_TIMEOUT:
195     dbg_event("—— DATA %d timeout\n", arg);
196     send_data_frame(arg);
197     break;
198
199 // 如何记录全局的ACK信号?
200 // 理论上应该对每个ack_timer记录对应编号
201 }
202
203 //发送窗口未满足且物理层已准备
204 if (send_nbuffered < NR_BUFS && phl_ready)
205     enable_network_layer();
206 else
207     disable_network_layer();
208 }
209
210 return EXIT_SUCCESS;
211 }

```

6.3 最终优化的 SR 协议

6.3.1 h 头文件

```

1 #ifndef __MYDATAINKBETTER_H_
2 #define __MYDATAINKBETTER_H_
3
4 #include "protocol.h"
5

```

```

6 #include <stdbool.h>
7 #define inc(k) (k == MAX_SEQ ? 0 : k + 1)
8 #define dec(k) (k == 0 ? MAX_SEQ : k - 1)
9 //重传时间应该从1500放宽至2000
10 #define DATA_TIMER (2000)
11 #define ACK_TIMER (400)
12
13 /* FRAME kind */
14 #define FRAME_DATA (1)
15 #define FRAME_ACK (2)
16 #define FRAME_NAK (3)
17 #define FRAME_SEQ_ACK (4)
18 #define MAX_SEQ (63)
19 #define NR_BUFS ((MAX_SEQ + 1) / 2)
20
21 static unsigned char in_buffer[NR_BUFS][PKT_LEN], out_buffer[NR_BUFS][PKT_LEN];
22 static bool phl_ready = false;
23 static bool no_nak = true;
24 static unsigned char send_nbuffered = 0; //发送窗口未被确认的单元数量
25 static unsigned char frame_nr = 0;      //发送帧编号,发送窗口上界,
26 //注意: 指的是还未发送的最优先编号
27 static unsigned char ack_expected = 0;   //发送窗口下界
28 static unsigned char ack_nr = NR_BUFS - 1; //接收窗口上界
29 static unsigned char frame_expected = 0;
30 //接收窗口下界
31 //首个未收到的数据单元序号, 可能在接收窗口内, 也可能在外
32 static bool arrived[MAX_SEQ + 1]; //从全序列号到窗口大小的映射 bitmap
33 //但在这里有冲突避免 接收窗口的确认以便重发NAK
34 static bool be_acked[MAX_SEQ + 1]; //发送窗口内被确认的
35 static void put_frame(unsigned char *frame, int len);
36 static void send_data_frame(unsigned char seq);
37 static void send_seq_ack_frame(unsigned char ack_num);
38 static void send_ack_frame(void);
39 // static void send_ack_frame(unsigned char ack_num);
40 static void send_nak_frame(void);
41
42 /*
43 DATA Frame
44 +-----+-----+-----+-----+-----+
45 | KIND(1) | ACK(1) | SEQ(1) | DATA(240~256) | CRC(4) |
46 +-----+-----+-----+-----+-----+
47
48 ACK Frame
49 +-----+-----+-----+
50 | KIND(1) | ACK(1) | CRC(4) |
51 +-----+-----+-----+
52
53 NAK Frame
54 +-----+-----+-----+
55 | KIND(1) | ACK(1) | CRC(4) |
56 +-----+-----+-----+
57 */
58 #endif // !_MYDATALINKBETTER_H_

```

6.3.2 C 源文件

```

1 #include "mydatalinkBetter.h"

```

```

2 #include "protocol.h"
3 #include <asm-generic/errno.h>
4 #include <stdlib.h>
5 #include <string.h>
6 typedef struct FRAME {
7     unsigned char kind;
8     unsigned char ack;
9     unsigned char seq;
10    unsigned char data[PKT_LEN];
11    unsigned int padding; // used for CRC detecting error
12 } frame;
13
14 /* 判断b是否在边界a,c之间 */
15 static bool between(unsigned char a, unsigned char b, unsigned char c) {
16     bool ret = ((a <= b) && (b <= c)) || ((c <= a) && (a <= b)) ||
17             ((b <= c) && (c <= a));
18     /*
19     if (ret) {
20         dbg_frame("\ntry to put frame %d to the network and works\
21 when the window size is [ %d , %d ]\n",
22                 b, a, c);
23     }*/
24     return ret;
25 }
26
27 // len字段保证类型统一
28 static void put_frame(unsigned char *frame, int len) {
29     *(unsigned int *) (frame + len) = crc32(frame, len);
30     send_frame(frame, len + 4);
31     phl_ready = false;
32 }
33
34 static void send_data_frame(unsigned char seq) {
35     frame f;
36     f.kind = FRAME_DATA;
37     //指定的累计确认ack字段
38     f.ack = dec(frame_expected); // i.e. dec(FRAME_EXPECTED)
39     f.seq = seq;
40     memcpy(f.data, out_buffer[seq % NR_BUFS], PKT_LEN);
41
42     dbg_frame("Send DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
43
44     put_frame((unsigned char *)&f, 3 + PKT_LEN);
45
46     //系统支持最大计数器 0到128
47     //注意timer是在何时被开启的, 此时数据帧是否被全部发送完毕?
48     start_timer(seq, DATA_TIMER);
49
50     //不再使用稍待确认和ack_timer
51     stop_ack_timer();
52 }
53
54 //累计确认ack和分别确认seq-ack
55 static void send_ack_frame(void) {
56     frame f;
57     f.kind = FRAME_ACK;
58
59     f.ack = dec(frame_expected);
60     dbg_frame("Send ACK %d\n", f.ack);

```

```

61
62     put_frame((unsigned char *)&f, 2);
63     //对于主动发送ack的类型处理
64     // stop_ack_timer();
65 }
66 static void send_seq_ack_frame(unsigned char ack_num) {
67     frame f;
68     f.kind = FRAME_SEQ_ACK;
69     f.ack = ack_num;
70     dbg_frame("Send SEQ ACK %d\n", f.ack);
71     put_frame((unsigned char *)&f, 2);
72 }
73
74 static void send_nak_frame(void) {
75     frame f;
76     f.kind = FRAME_NAK;
77     f.ack = dec(frame_expected);
78     dbg_frame("Send NAK %d\n", f.ack);
79     //顺序是否有要求?,应该无影响
80     no_nak = false;
81     put_frame((unsigned char *)&f, 2);
82     stop_ack_timer();
83 }
84
85 //可以将main函数中的功能进一步划分模块
86 int main(int argc, char *argv[]) {
87     memset(in_buffer, 0, sizeof in_buffer);
88     memset(out_buffer, 0, sizeof out_buffer);
89     memset(arrived, false, sizeof arrived);
90     memset(be_acked, false, sizeof be_acked);
91     // unsigned char zeroArr[PKT_LEN];
92     // memset(zeroArr, 0, sizeof zeroArr);
93     int event, arg;
94     int len = 0;
95     frame f;
96     protocol_init(argc, argv);
97     lprintf("Designed by Wang Zixuan, build: " __DATE__ " " __TIME__
98           "\n");
99
100     disable_network_layer();
101     for (;;) {
102         event = wait_for_event(&arg);
103         switch (event) {
104             /* 准备数据分组 */
105             case NETWORK_LAYER_READY:
106                 get_packet(out_buffer[frame_nr % NR_BUFS]);
107                 send_nbuffered++;
108                 send_data_frame(frame_nr);
109                 be_acked[frame_nr] = false; /* 更新发送窗口状态*/
110                 // 唯一的 发送窗口的扩展
111                 frame_nr = inc(frame_nr);
112                 break;
113             case PHYSICAL_LAYER_READY:
114                 phl_ready = true;
115                 break;
116
117             case FRAME_RECEIVED:
118                 //坏帧
119                 len = recv_frame((unsigned char *)&f, sizeof(f));

```

```

120     if (len < 6 || crc32((unsigned char *)&f, len) != 0) {
121         dbg_event("**** Receiver Error, Bad CRC Checksum\n");
122         if (no_nak) {
123             send_nak_frame();
124         }
125         break;
126     }
127
128     if (f.kind == FRAME_DATA) {
129
130         /* 先处理接收逻辑*/
131         dbg_frame("Recv DATA %d %d, ID %d\n", f.seq, f.ack, *(short *)f.data);
132         //如果失序, 首先请求重传期望的帧
133         if (f.seq != frame_expected && no_nak &&
134             between(frame_expected, f.seq, ack_nr)) {
135             send_nak_frame();
136             //如果传输过程中出错, 依靠发送方的DATA_TIMER定时器进行数据重传
137         }
138         if (!between(frame_expected, f.seq, ack_nr))
139             send_seq_ack_frame(f.seq);
140
141         if (between(frame_expected, f.seq, ack_nr) &&
142             (frame_expected != ack_nr) && arrived[f.seq] == false) {
143             //拷贝需要时间!! 事件响应过程需要时间
144             //但在这里是单线程, 处理较简单
145             if (memcpy(in_buffer[f.seq % NR_BUFS], f.data, PKT_LEN) != NULL) {
146                 // len在这里固定为PKT_LEN
147                 dbg_frame("\n%d frame copy to the buffer %d\n", f.seq,
148                     *(short *)in_buffer[f.seq % NR_BUFS]);
149                 arrived[f.seq] = true;
150             }
151             if (f.seq != frame_expected)
152                 send_seq_ack_frame(f.seq);
153             else
154                 start_ack_timer(ACK_TIMER);
155             //开始滑动窗口
156             while (arrived[frame_expected] ==
157                 true //&&
158                     // memcmp(in_buffer[frame_expected % NR_BUFS],
159                     // zeroArr,
160                     // PKT_LEN) != 0
161             ) {
162                 no_nak = true;
163                 dbg_frame("\nWant to put %d to the network and ready to put frame "
164                     "which ID is %d to the network layer\n",
165                     when the window size is [ %d , %d ]\n ",
166                     frame_expected,
167                     *(short *)in_buffer[frame_expected % NR_BUFS],
168                     frame_expected, ack_nr);
169                 put_packet(in_buffer[frame_expected % NR_BUFS], PKT_LEN);
170                 memset(in_buffer[frame_expected % NR_BUFS], 0, PKT_LEN);
171                 arrived[frame_expected] = false;
172                 // dbg_frame("\nMake %d to be false in arrived\n",
173                 // frame_expected);
174                 frame_expected = inc(frame_expected);
175                 ack_nr = inc(ack_nr);
176             }
177         }
178     }

```

```

179     if (f.kind == FRAME_ACK) {
180         if (be_acked[f.ack] == false) {
181             be_acked[f.ack] = true;
182             stop_timer(f.ack);
183         }
184         dbg_frame("Recv ACK %d\n", f.ack);
185     }
186
187     if (f.kind == FRAME_NAK) {
188         dbg_frame("Recv NAK %d\n", f.ack);
189         dbg_frame("\nthe send window size is [ %d , %d]\n", ack_expected,
190                 dec(frame_nr));
191         if (send_nbuffered != 0 &&
192             between(ack_expected, inc(f.ack), dec(frame_nr))) {
193             send_data_frame(inc(f.ack));
194         }
195     }
196     if (f.kind == FRAME_SEQ_ACK) {
197         dbg_frame("Confirm recv frame %d\n", f.ack);
198         stop_timer(f.ack);
199         if (between(frame_expected, f.ack, ack_nr) && be_acked[f.ack] == false)
200             be_acked[f.ack] = true;
201     }
202     /* 处理发送逻辑*/
203
204     //根据确认帧中的信息避免冗余数据帧的发送
205     if ((f.kind == FRAME_ACK || f.kind == FRAME_DATA) && send_nbuffered &&
206         between(ack_expected, f.ack, dec(frame_nr))) {
207         for (unsigned char i = ack_expected; i != inc(f.ack); i = inc(i)) {
208             if (be_acked[i] == false) {
209                 be_acked[i] = true;
210                 stop_timer(i);
211             }
212         }
213     }
214     while (be_acked[ack_expected] == true && send_nbuffered) {
215         // 确保不被错误接受
216         // be_acked[ack_expected] = false;
217         ack_expected = inc(ack_expected);
218         send_nbuffered--;
219     }
220     break;
221 case DATA_TIMEOUT:
222     dbg_event("—— DATA %d timeout\n", arg);
223     send_data_frame(arg);
224     break;
225
226     // 如何记录全局的ACK信号?
227     // 理论上应该对每个ack_timer记录对应编号
228 case ACK_TIMEOUT:
229     dbg_event("—— ACK timeout\n");
230     send_ack_frame();
231     break;
232 }
233
234 //发送窗口未满足且物理层已准备
235 if (send_nbuffered < NR_BUFS && phl_ready)
236     enable_network_layer();
237 else

```

```

238     disable_network_layer();
239 }
240
241 return EXIT_SUCCESS;
242 }

```

6.4 gobackn 协议

6.4.1 h 头文件

```

1  /* FRAME kind */
2  typedef unsigned char FRAME_KIND;
3  #define FRAME_DATA 0
4  #define FRAME_ACK 1
5  #define FRAME_NAK 2
6
7  #define NONE "\e[0m" //清除颜色，即之后的打印为正常输出，之前的不受影响
8  #define BLACK "\e[0;30m" //深黑
9  #define L_BLACK "\e[1;30m" //亮黑，偏灰褐
10 #define RED "\e[0;31m" //深红，暗红
11 #define L_RED "\e[1;31m" //鲜红
12 #define GREEN "\e[0;32m" //深绿，暗绿
13 #define L_GREEN "\e[1;32m" //鲜绿
14 #define BROWN "\e[0;33m" //深黄，暗黄
15 #define YELLOW "\e[1;33m" //鲜黄
16 #define BLUE "\e[0;34m" //深蓝，暗蓝
17 #define L_BLUE "\e[1;34m" //亮蓝，偏白灰
18 #define PURPLE "\e[0;35m" //深粉，暗粉，偏暗紫
19 #define L_PURPLE "\e[1;35m" //亮粉，偏白灰
20 #define CYAN "\e[0;36m" //暗青色
21 #define L_CYAN "\e[1;36m" //鲜亮青色
22 #define GRAY "\e[0;37m" //灰色
23 #define WHITE "\e[1;37m" //白色，字体粗一点，比正常大，比bold小
24 #define BOLD "\e[1m" //白色，粗体
25 #define UNDERLINE "\e[4m" //下划线，白色，正常大小
26 #define BLINK "\e[5m" //闪烁，白色，正常大小
27 #define REVERSE "\e[7m" //反转，即字体背景为白色，字体为黑色
28 #define HIDE "\e[8m" //隐藏
29 #define CLEAR "\e[2J" //清除
30 #define CLRLINE "\r\e[K" //清除行

```

6.4.2 C 源文件

```

1  #include "gobackn.h"
2  #include "protocol.h"
3  #include <stdbool.h>
4  #include <string.h>
5
6  #define MAX_SEQ 31
7  #define DATA_TIMER 11600
8
9  typedef unsigned char seq_nr;
10 typedef struct frame {
11     FRAME_KIND kind;
12     seq_nr ack;
13     seq_nr seq;

```

```

14 unsigned char data[PKT_LEN];
15 unsigned int padding;
16 } FRAME;
17
18 static bool no_nak = true;
19 static int phl_ready = 0;
20
21 static int between(seq_nr a, seq_nr b, seq_nr c) {
22     return ((a <= b && b < c) || (c < a && a <= b) || (b < c && c < a));
23 }
24
25 static void put_frame(seq_nr *frame, int len) {
26     *((unsigned int *) (frame + len)) = crc32(frame, len);
27     send_frame(frame, len + 4);
28     phl_ready = 0;
29 }
30
31 static void send_data_frame(FRAME_KIND fk, seq_nr frame_seq, seq_nr ack_seq,
32                             unsigned char out_buf[][PKT_LEN]) {
33     FRAME frame;
34     frame.kind = fk; // 帧类型
35     frame.seq = frame_seq; // 帧序号
36     frame.ack = (ack_seq + MAX_SEQ) % (MAX_SEQ + 1); // 捎带ACK序号
37     if (fk == FRAME_DATA) {
38         memcpy(frame.data, out_buf[frame_seq], PKT_LEN);
39         dbg_frame("Send DATA %d with ACK %d, ID %d"
40                 "\n",
41                 frame.seq, frame.ack, *((short *) frame.data);
42         put_frame((seq_nr *) &frame, 3 + PKT_LEN);
43         start_timer(frame_seq, DATA_TIMER); // 启动数据定时器
44     } else if (fk == FRAME_NAK) {
45         no_nak = false;
46         dbg_frame(L_BLUE "Send NAK %d" NONE "\n", frame.seq);
47         put_frame((unsigned char *) &frame, 2);
48     }
49     stop_ack_timer();
50 }
51
52 static void inc(seq_nr *seq) {
53     *seq = (*seq + 1) % (MAX_SEQ + 1); // 数据指针后移
54 }
55
56 int main(int argc, char **argv) {
57     seq_nr out_buf[MAX_SEQ][PKT_LEN];
58     seq_nr nbuffered = 0;
59     seq_nr ack_expected = 0;
60     seq_nr next_frame_to_send = 0;
61     seq_nr frame_expected = 0;
62
63     int event, arg;
64     FRAME f;
65     int len = 0;
66
67     protocol_init(argc, argv);
68     lprintf("Reconstruct by wxz, build: " __DATE__ " " __TIME__
69            "\n");
70
71     enable_network_layer();
72

```

```

73  for (;;) {
74      event = wait_for_event(&arg);
75
76      switch (event) {
77          /* 网络层就绪 */
78          case NETWORK_LAYER_READY:
79              /* 从网络层拿包 */
80              get_packet(out_buf[next_frame_to_send]);
81              ++nbuffered;
82              /* 发送数据包 */
83              send_data_frame(FRAME_DATA, next_frame_to_send, frame_expected, out_buf);
84              inc(&next_frame_to_send);
85              break;
86
87          /* 物理层就绪 */
88          case PHYSICAL_LAYER_READY:
89              phl_ready = 1;
90              break;
91
92          /* 收到数据包 */
93          case FRAME_RECEIVED:
94              len = recv_frame((seq_nr *)&f, sizeof f);
95              /* CRC 校验 */
96              if (len < 5 || crc32((seq_nr *)&f, len) != 0) {
97                  dbg_event(L_RED "**** Receiver Error, Bad CRC Checksum" NONE "\n");
98                  if (no_nak) {
99                      /* 校验错误且未发送NAK */
100                      send_data_frame(FRAME_NAK, frame_expected, 0, 0);
101                  }
102                  break;
103              }
104              /* 数据包 */
105              if (f.kind == FRAME_DATA) {
106                  if (f.seq == frame_expected) {
107                      dbg_frame("Recv DATA %d with piggyback ACK %d, ID %d\n", f.seq, f.ack,
108                              *(short *)f.data);
109                      put_packet(f.data, len - 7);
110                      no_nak = true;
111                      inc(&frame_expected);
112                  } else if (no_nak) {
113                      /* 序号有误, 发送NAK */
114                      send_data_frame(FRAME_NAK, frame_expected, 0, 0);
115                  }
116              }
117              /* NAK包 */
118              if (f.kind == FRAME_NAK) {
119                  dbg_frame(L_PURPLE "Recv NAK %d" NONE "\n", ack_expected);
120                  next_frame_to_send = ack_expected;
121                  dbg_frame("nbuffered: %d\n", nbuffered);
122                  for (seq_nr i = 1; i <= nbuffered; i++) { // 0 or 1?
123                      dbg_frame("REsend DATA %d with ACK %d\n", next_frame_to_send,
124                              (frame_expected + MAX_SEQ) % (MAX_SEQ + 1));
125                      send_data_frame(FRAME_DATA, next_frame_to_send, frame_expected,
126                                      out_buf);
127                      inc(&next_frame_to_send);
128                  }
129                  break;
130              }
131              /* 处理捎带ACK */

```

```
132     while (between(ack_expected, f.ack, next_frame_to_send)) {
133         --nbuffered;
134         stop_timer(ack_expected);
135         inc(&ack_expected);
136     }
137     break;
138
139     /* 数据定时器超时 */
140     case DATA_TIMEOUT:
141         next_frame_to_send = ack_expected;
142         dbg_event(YELLOW "----- DATA %d timeout" NONE "\n", arg);
143         dbg_frame("nbuffered: %d\n", nbuffered);
144         for (seq_nr i = 1; i <= nbuffered; i++) { // 0 or 1?
145             send_data_frame(FRAME_DATA, next_frame_to_send, frame_expected,
146                             out_buf);
147             inc(&next_frame_to_send);
148         }
149     }
150
151     if (nbuffered < MAX_SEQ && phl_ready)
152         enable_network_layer();
153     else
154         disable_network_layer();
155 }
156 }
```

参考文献

- [1] 小林 coding. 面试官: 换人! 他连 tcp 这几个参数都不懂, 2020.
- [2] 群星的尽头. Crc (循环冗余校验码) 简介与实现解析, 2020.